Contrôle d'un Jeu Vidéo par Gestes de la Main

Vision par Ordinateur —

Cours d'Initiation
Master 2 Mathématiques, 2025–2026

Webcam $\longrightarrow$ MediaPipe $\longrightarrow$ CNN $\longrightarrow$ WebSocket $\longrightarrow$ Jeu

Auteurs Christ Bohou
Djafarou Oulare

Dépôt `https://github.com/bohouchrist/projet_computer_vision`

Date Mars 2026

Ce rapport présente un système complet de reconnaissance de gestes en temps réel appliqué au contrôle du jeu Space Invaders dans un navigateur web. Il documente quatre approches expérimentées, les choix architecturaux et les résultats obtenus.

Table des matières

1 Introduction et Contexte **3**
1.1 Objectifs techniques . . . 3
1.2 Gestes retenus . . . 3

2 Architecture Globale du Système **3**
2.1 Vue d'ensemble . . . 3
2.2 MediaPipe HandLandmarker . . . 4
2.3 Architecture logicielle — threading Python . . . 4
2.4 Hold Timer — serveur Node.js . . . 5

3 Collecte et Préparation des Données **5**
3.1 Approche 1 — Images brutes 64×64 (Christ Bohou) . . . 5
3.2 Approche 2 — Landmarks normalisés (Christ Bohou) . . . 6
3.3 Approche 3 — Dataset Kaggle (Djafarou Oulare) . . . 7

4 Modèles de Classification **7**
4.1 Modèle 1 — CNN sur images brutes (Christ Bohou) . . . 8
4.1.1 Architecture . . . 8
4.1.2 Data Augmentation . . . 8
4.1.3 Callbacks et régularisation . . . 9
4.1.4 Résultats . . . 9
4.2 Modèle 2 — MLP sur images Kaggle (Djafarou Oulare) . . . 10
4.2.1 Architecture . . . 10
4.3 Modèle 3 — CNN sur images Kaggle (Djafarou Oulare) . . . 10
4.3.1 Architecture . . . 10
4.4 Modèle 4 — Réseau Dense sur Landmarks (Christ Bohou) — Solution retenue . . . 11
4.4.1 Principe et motivation . . . 11
4.4.2 Architecture complète . . . 12
4.4.3 Entraînement . . . 12
4.4.4 Résultats . . . 13

5 Comparaison des Approches **14**
5.1 Leçon principale : la représentation avant le modèle . . . 14

6 Intégration Temps Réel et Déploiement **15**
6.1 Pipeline d'inférence . . . 15
6.2 Cooldowns anti-spam . . . 16
6.3 Correspondance des touches . . . 16

7 Résultats, Limites et Analyse Critique **16**
7.1 Performance en conditions réelles 16
7.2 Limites identifiées 16

8 Perspectives d'Amélioration **17**
8.1 Court terme — Classes composites pour tirer en se déplaçant 17
8.2 Court terme — Robustesse FIRE deux mains 17
8.3 Moyen terme — LSTM pour gestes dynamiques 17
8.4 Long terme — Deux mains simultanées 18

9 Conclusion **18**

A Structure du Dépôt **19**

B Guide de Lancement **19**
B.1 Prérequis 19
B.2 Lancement automatique (recommande) 19
B.3 Lancement manuel 19
B.4 Test du modèle seul 20

1 Introduction et Contexte

Ce projet a pour objectif de concevoir un système de **reconnaissance de gestes en temps réel** permettant de contrôler le jeu classique *Space Invaders* directement par des mouvements de la main devant une webcam standard.

Il s'inscrit dans le cadre d'un cours d'initiation à la vision par ordinateur en Master 2 Mathématiques, où les compétences clés à démontrer sont : la collecte de données visuelles, la construction et l'entraînement de réseaux de neurones, et leur intégration dans un pipeline temps réel.

1.1 Objectifs techniques

1. Détecter et classifier 5 gestes de la main en temps réel à au moins 25 fps.
2. Transmettre les commandes au jeu via WebSocket avec une latence inférieure à 100 ms.
3. Atteindre une précision de classification suffisante pour une jouabilité fluide.

1.2 Gestes retenus

Dictionnaire de gestes — version finale

Photo	Geste	Description	Commande
	LEFT	Main droite, index pointé vers la gauche	`LEFT`
	RIGHT	Main gauche, index pointé vers la droite	`RIGHT`
	FIRE	Poing fermé (knuckles face caméra)	`FIRE`
	ENTER	Paume ouverte face caméra	`ENTER`
	NEUTRAL	Main au repos (aucune commande)	—

2 Architecture Globale du Système

2.1 Vue d'ensemble

Le système repose sur quatre processus indépendants communiquant en temps réel :

FIGURE 1 – Pipeline complet de contrôle gestuel en temps réel.

2.2 MediaPipe HandLandmarker

MediaPipe HandLandmarker est un modèle de détection de la main pré-entraîné par Google, basé sur une architecture MobileNet légère. Il opère en deux étapes :

1. **BlazePalm** : détecte la région de la main dans l'image en moins de 5 ms.
2. **Estimation des landmarks** : prédit les coordonnées (x, y, z) de **21 points anatomiques** (poignet, phalanges proximal/médial/distal de chaque doigt) avec une incertitude inférieure à 2 mm.

En mode `VIDEO`, un tracker léger entre les frames évite la ré-détection à chaque image, permettant un traitement à 30+ fps sur CPU sans GPU.

Paramètre	**Valeur**
min_hand_detection_confidence	0.6
min_hand_presence_confidence	0.5
min_tracking_confidence	0.5
num_hands	1

2.3 Architecture logicielle — threading Python

Le script principal `5_jouer_avec_cnn.py` utilise une architecture bi-thread pour éviter que le WebSocket bloque la capture vidéo :

Listing 1 – Communication inter-threads par file de messages (extrait).

```python
import queue, threading, json, websocket

commande_queue = queue.Queue(maxsize=20)

def thread_websocket(ws_uri):
    """Thread dedie a l'envoi WebSocket - ne bloque pas la video."""
    ws = websocket.WebSocket()
    ws.connect(ws_uri)
    while True:
        msg = commande_queue.get(timeout=0.5)
        ws.send(json.dumps(msg))

# Lancement du thread WS en arriere-plan
t = threading.Thread(target=thread_websocket, args=(WS_URI,),
    daemon=True)
```

```python
t.start()

# Thread principal : webcam + MediaPipe + CNN
while True:
    ret, frame = cap.read()
    ...
    commande_queue.put_nowait({'type': geste})  # non-bloquant
```

2.4 Hold Timer — serveur Node.js

Le jeu JavaScript attend des événements `keydown`/`keyup` continus, alors que Python envoie des commandes discrètes (30 fois/seconde). Le serveur Node.js résout ce décalage avec un **hold timer de 180 ms** :

Listing 2 – Hold timer Node.js ? maintient la touche enfonc ?e tant que la commande arrive.

```javascript
const HOLD_TIMEOUT = 180; // ms
const timers = {};

function handleCommand(type) {
  if (timers[type]) clearTimeout(timers[type]); // remet le compteur a zero
  else sendKeyDown(KEYMAP[type]);               // premiere arrivee = keydown

  timers[type] = setTimeout(() => {
    sendKeyUp(KEYMAP[type]);                    // plus de signal = keyup
    delete timers[type];
  }, HOLD_TIMEOUT);
}
```

3 Collecte et Préparation des Données

3.1 Approche 1 — Images brutes 64×64 (Christ Bohou)

Le script `1_collecter_images.py` capture automatiquement une image toutes les 300 ms via OpenCV pendant que l'utilisateur maintient un geste. Chaque image est redimensionnée en 64×64 pixels RGB et sauvegardée en JPEG.

Listing 3 – Capture automatique avec d ?lai fixe (extrait).

```python
AUTO_DELAY = 0.3  # secondes entre chaque capture automatique
IMG_SIZE   = 64

```

```
ret, frame = cap.read()
frame = cv2.flip(frame, 1)  # miroir horizontal (webcam)
img = cv2.resize(frame, (IMG_SIZE, IMG_SIZE))
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
nom = f"{classe}_{compteur:04d}.jpg"
cv2.imwrite(os.path.join(dossier_classe, nom), img_rgb)
```

Classe	Images	Proportion
LEFT	502	19.9%
RIGHT	516	20.5%
FIRE	520	20.6%
ENTER	520	20.6%
NEUTRAL	463	18.4%
Total	**2 521**	100%

3.2 Approche 2 — Landmarks normalisés (Christ Bohou)

Le script `3_collecter_landmarks.py` utilise MediaPipe en mode `VIDEO` pour extraire les 21 landmarks par frame et les sauvegarde en CSV après normalisation.

Normalisation des landmarks. Pour chaque frame, le vecteur brut $\mathbf{p} \in \mathbb{R}^{21\times 3}$ est transformé par :

$$\tilde{\mathbf{p}}_i = \frac{\mathbf{p}_i - \mathbf{p}_0}{\max_j \|\mathbf{p}_j - \mathbf{p}_0\|_\infty} \tag{1}$$

où $\mathbf{p}_0$ est la position du poignet (landmark 0). Le résultat $\tilde{\mathbf{p}} \in [-1, 1]^{21\times 3}$, aplati en un vecteur de 63 valeurs, est invariant à la position, l'échelle et la taille de la main.

Listing 4 – Normalisation des landmarks ? identique ? l'entra ?nement et ? l'inf ?rence.

```
def normaliser(landmarks):
    pts = np.array([[lm.x, lm.y, lm.z] for lm in landmarks],
                   dtype=np.float32)
    pts -= pts[0]                # centrage sur le poignet
    s = np.max(np.abs(pts))
    if s > 0:
        pts /= s                 # mise a l'echelle dans [-1, 1]
    return pts.flatten()         # vecteur de 63 valeurs
```

Environ **2 100 échantillons** ont été collectés à raison d'une capture automatique toutes les 300 ms, soit environ 7 minutes de collecte par classe. La même normalisation

est appliquée à l'identique lors de l'inférence temps réel, garantissant la cohérence entre entraînement et déploiement.

3.3 Approche 3 — Dataset Kaggle (Djafarou Oulare)

En parallèle, un dataset externe Kaggle (*Hand Navigation Landmarks*, harisudhan411) a été exploité. Il contient 8 000 images 180×180 en 4 classes (left, right, up, down) photographiées sur fond blanc uniforme.

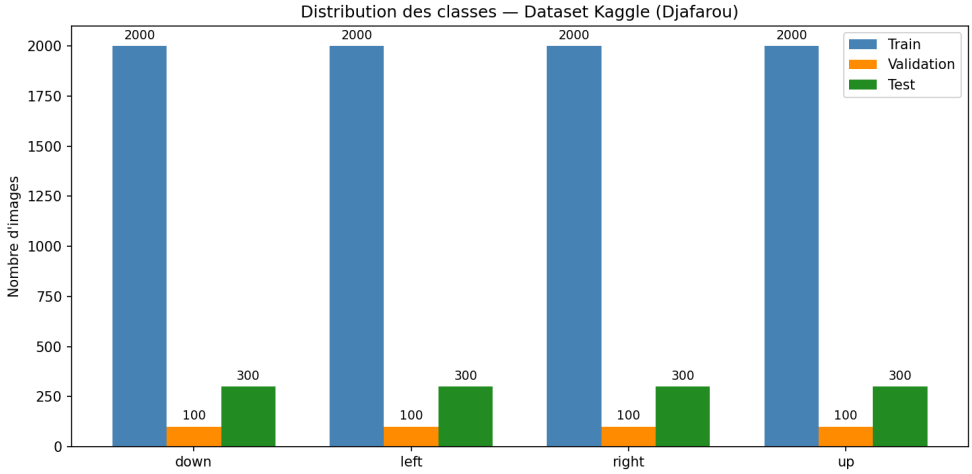

FIGURE 2 – Distribution des classes du dataset Kaggle (Djafarou Oulare). Jeu parfaitement équilibré : 2 000 images par classe en train, 100 en validation, 300 en test.

Prétraitement (Djafarou). Les images sont chargées avec `tf.keras.utils.image_dataset_from` normalisées par division par 255 (`tf.cast(image, tf.float32)/255.0`), puis les labels sont encodés en vecteurs one-hot :

Listing 5 – Pr ?traitement du dataset Kaggle (Djafarou).

```
def normaliser(image, label):
    return tf.cast(image, tf.float32) / 255.0, label

def one_hot_label(image, label):
    return image, tf.one_hot(label, num_classes=4)

train_ds = train_ds.map(normaliser).map(one_hot_label)
val_ds   = val_ds.map(normaliser).map(one_hot_label)
test_ds  = test_ds.map(normaliser).map(one_hot_label)
```

4 Modèles de Classification

Quatre modèles ont été expérimentés au total : deux par Christ Bohou (CNN images brutes, réseau dense sur landmarks) et deux par Djafarou Oulare (MLP et CNN sur

images Kaggle).

4.1 Modèle 1 — CNN sur images brutes (Christ Bohou)

4.1.1 Architecture

Un CNN à 3 blocs convolutifs avec data augmentation en ligne a été construit.

Listing 6 – Architecture CNN 3 blocs (2_entrainer_cnn.py).

```
modele = models.Sequential([
    # Bloc 1 - detection de contours simples
    layers.Conv2D(32, (3,3), activation='relu', padding='same',
                  input_shape=(64, 64, 3)),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D(2, 2),
    layers.Dropout(0.25),

    # Bloc 2 - detection de formes (doigts, paume)
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D(2, 2),
    layers.Dropout(0.25),

    # Bloc 3 - patterns complexes (configuration de la main)
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.Dropout(0.25),

    # Tete de classification
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(5, activation='softmax'),   # 5 classes
])
```

Total paramètres entraînables : 2 240 037.

4.1.2 Data Augmentation

Pour compenser la faible taille du dataset (2 521 images), une augmentation de données en ligne est appliquée à chaque batch pendant l'entraînement :

Listing 7 – Pipeline d'augmentation de donn ?es.

```
augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),      # miroir horizontal
    layers.RandomRotation(0.15),          # rotation +/- 15%
    layers.RandomZoom(0.15),              # zoom +/- 15%
    layers.RandomBrightness(0.2),         # luminosite +/- 20%
    layers.RandomContrast(0.2),           # contraste +/- 20%
])
```

4.1.3 Callbacks et régularisation

Listing 8 – Callbacks d'entra ?nement.

```
cb_list = [
    callbacks.EarlyStopping(
        monitor='val_accuracy', patience=8,
        restore_best_weights=True),           # arret si stagnation
    callbacks.ModelCheckpoint(
        filepath=MODEL_PATH,
        monitor='val_accuracy', save_best_only=True),
    callbacks.ReduceLROnPlateau(
        monitor='val_loss', factor=0.5,
        patience=4, min_lr=1e-6),             # reduction lr si
            val_loss stagne
]
```

4.1.4 Résultats

Résultat — CNN images brutes (Christ Bohou)

Meilleure val_accuracy : 34.3% (époque 14 sur 22 avant early stopping).
Précision chance aléatoire : 20% (5 classes équiréparties).
Le modèle n'apprend pas à généraliser au-delà du hasard.

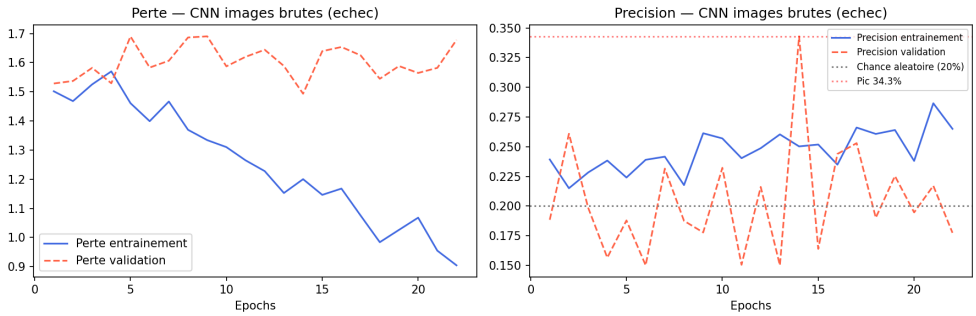

FIGURE 3 – Courbes d'entraînement du CNN images brutes. La précision de validation stagne autour de 20% (niveau aléatoire) avec un pic isolé à 34.3% à l'époque 14. Aucune convergence stable n'est atteinte.

Analyse de l'échec. Le CNN apprend à reconnaître le fond de la pièce autant que la forme de la main. Avec 2 521 images collectées dans un seul environnement d'éclairage, le modèle mémorise des artefacts contextuels (couleur du mur, ombres) plutôt que la géométrie de la main. Il en résulte une généralisation quasi nulle.

4.2 Modèle 2 — MLP sur images Kaggle (Djafarou Oulare)

4.2.1 Architecture

Un perceptron multicouche (MLP) minimaliste est construit sur les images 180×180×3 aplaties en vecteur 1D :

Listing 9 – Architecture MLP sur images 180 ?180 (Djafarou).

```
model = keras.Sequential([
    layers.Flatten(input_shape=(180, 180, 3)),  # 97200 entrees
    layers.Dense(128, activation='relu'),
    layers.Dense(4, activation='softmax')       # 4 classes
])
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
historique = model.fit(train_ds, validation_data=val_ds, epochs
    =100)
```

Paramètres : 12 457 348. La couche Flatten génère un vecteur de 97 200 dimensions, ce qui rend le modèle massivement surparamétré pour 4 classes.

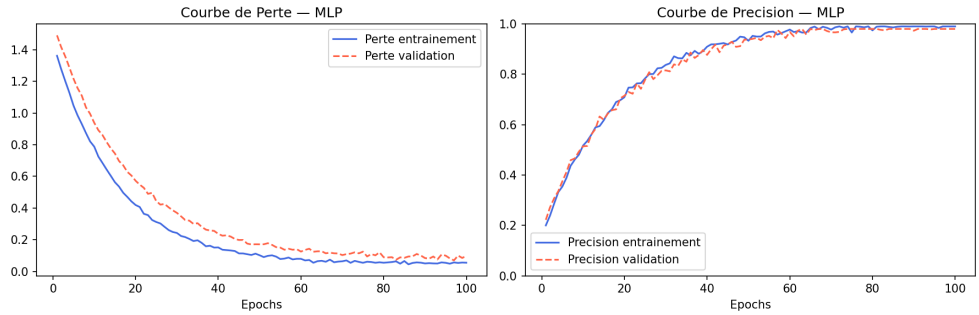

FIGURE 4 – Courbes d'entraînement du MLP sur le dataset Kaggle (Djafarou). Convergence progressive sur 100 epochs.

4.3 Modèle 3 — CNN sur images Kaggle (Djafarou Oulare)

4.3.1 Architecture

Listing 10 – Architecture CNN Kaggle (Djafarou).

```
model_cnn = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(180,180,3)),
```

```
    MaxPooling2D(pool_size=(2,2)),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(4, activation='softmax')              # 4 classes
])
model_cnn.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
history_cnn = model_cnn.fit(train_ds, validation_data=val_ds,
    epochs=10)
```

Paramètres : 15 167 044 dont 15 147 136 dans la seule couche Dense (118 336 → 128), ce qui révèle un goulot d'étranglement architectural.

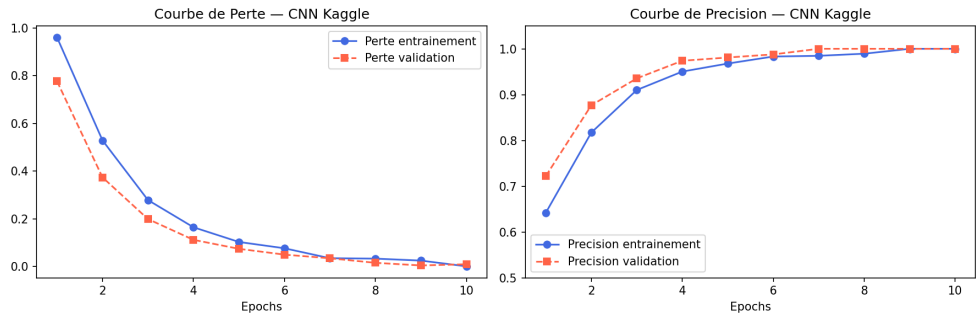

FIGURE 5 – Courbes d'entraînement du CNN Kaggle (Djafarou). Convergence rapide vers 100% en 10 epochs sur le dataset à fond blanc.

Limites communes des approches pixel

Les trois modèles travaillant sur des pixels bruts souffrent du même problème : ils apprennent des *corrélations spurieuses* entre le fond et la classe. Un modèle entraîné sur fond blanc (Kaggle) prédit **«up» à 99.8%** sur n'importe quelle image aléatoire — il n'a pas appris la forme de la main mais les statistiques du fond blanc.

4.4 Modèle 4 — Réseau Dense sur Landmarks (Christ Bohou) — Solution retenue

4.4.1 Principe et motivation

Au lieu de traiter les pixels, on extrait d'abord la représentation géométrique de la main (landmarks MediaPipe), puis on entraîne un réseau dense sur ces coordonnées normalisées. La représentation obtenue est :

— **invariante au fond** : aucune information de pixel
— **invariante à la luminosité et à la couleur de peau**
— **invariante à la taille** et à la distance à la caméra (normalisation)
— **compacte** : 63 réels au lieu de $64 \times 64 \times 3 = 12\,288$

4.4.2 Architecture complète

Listing 11 – Architecture du r ?seau dense sur landmarks (4_entrainer_landmarks.py).

```
model = keras.Sequential([
    keras.Input(shape=(63,)),                # 21 points x 3 coords

    keras.layers.Dense(128, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.3),

    keras.layers.Dense(128, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.3),

    keras.layers.Dense(64, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.2),

    keras.layers.Dense(5, activation='softmax'),    # 5 classes
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'],
)
```

Total paramètres : 33 925 — soit 66 fois moins que le CNN images.

4.4.3 Entraînement

Listing 12 – Boucle d'entra ?nement avec callbacks.

```
callbacks = [
    keras.callbacks.ModelCheckpoint(
        MODEL_PATH, monitor='val_accuracy', save_best_only=True),
    keras.callbacks.EarlyStopping(
        monitor='val_accuracy', patience=15,
        restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss', factor=0.5, patience=5),
```

```
]

history = model.fit(
    X_train, y_train,
    epochs=50, batch_size=32,
    validation_data=(X_val, y_val),
    callbacks=callbacks,
)
```

Le modèle converge en **13 époques** sur les 50 max autorisées, avec un temps d'entraînement total inférieur à 30 secondes sur CPU.

4.4.4 Résultats

Résultat — Réseau dense sur landmarks (solution déployée)

Meilleure val_accuracy : 99.6% — 2 erreurs sur 501 échantillons.
Convergence à l'époque 13. Temps total d'entraînement : < 30 s sur CPU.

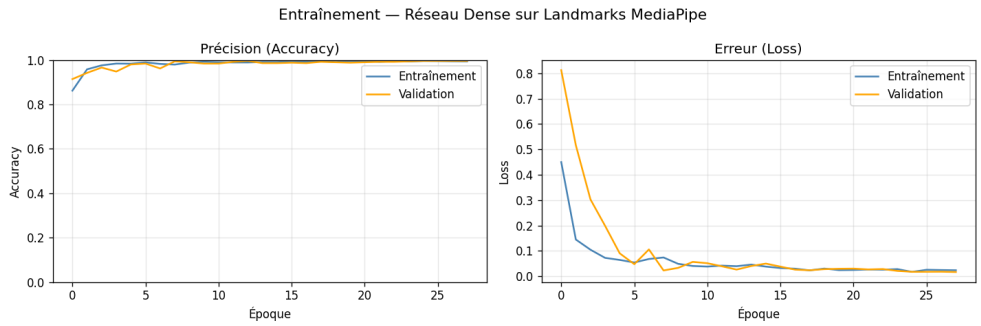

FIGURE 6 – Courbes d'entraînement du réseau dense sur landmarks. La convergence est rapide (époque 13) et stable, sans sur-apprentissage. La validation suit l'entraînement de près.

Matrice de confusion (501 échantillons de validation).

Réel \ Prédit	ENTER	FIRE	LEFT	NEUTRAL	RIGHT	**Total**
ENTER	**100**	0	0	0	0	100
FIRE	0	**100**	0	0	0	100
LEFT	0	0	**100**	0	0	100
NEUTRAL	0	1	1	**99**	0	101
RIGHT	0	0	0	0	**100**	100
Total prédit	100	101	101	99	100	501

Les 2 erreurs concernent uniquement la classe NEUTRAL, confondue une fois avec FIRE et une fois avec LEFT. Explication géométrique : une main en position de repos partielle (doigts semi-fléchis) produit un vecteur de landmarks proche d'un poing ou d'un index semi-étendu.

5 Comparaison des Approches

Critère	CNN images	MLP Kaggle	CNN Kaggle	Dense landmarks
Val. accuracy	34.3%	∼90%	∼100%	**99.6%**
Perf. en conditions réelles	Mauvaise	Mauvaise	Mauvaise	**Bonne**
Paramètres	2.24 M	12.5 M	15.2 M	**33 925**
Temps/epoch (CPU)	∼16 s	∼60 s	∼45 s	**$<$ 1 s**
Données nécessaires	>50 k	8 000 (ext.)	8 000 (ext.)	**∼2 000**
Sensibilité au fond	Critique	Critique	Critique	**Nulle**
Invariant taille main	Non	Non	Non	**Oui**
Dataset propre	Oui	Non	Non	**Oui**

Table 1 – Comparaison des quatre approches expérimentées.

5.1 Leçon principale : la représentation avant le modèle

La comparaison la plus révélatrice est celle entre le CNN images brutes (34.3%) et le réseau dense sur landmarks (99.6%). Ces deux modèles opèrent dans des conditions identiques (même pièce, même webcam, mêmes gestes), mais la représentation de l'entrée est radicalement différente :

	CNN images	Dense landmarks
Dimensionnalité entrée	$64 \times 64 \times 3 = 12\,288$	63
Réduction	$\div 195$	
Paramètres modèle	2 240 037	33 925
Réduction	$\div 66$	
Val. accuracy	34.3%	**99.6%**

Ce résultat illustre un principe fondamental de l'apprentissage automatique : *une bonne représentation (encodant uniquement les informations pertinentes) vaut davantage qu'un modèle plus grand ou plus complexe.*

6 Intégration Temps Réel et Déploiement

6.1 Pipeline d'inférence

La boucle d'inférence principale traite chaque frame en 3 étapes :

Listing 13 – Boucle d'inf ?rence temps r ?el (5_jouer_avec_cnn.py).

```python
SEUIL_CONFIANCE = 0.70   # seuil pour envoyer une commande

while True:
    ret, frame = cap.read()
    frame = cv2.flip(frame, 1)      # miroir webcam

    # 1. Extraction des landmarks MediaPipe
    rgb    = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    mp_img = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
    result = landmarker.detect_for_video(mp_img, timestamp_ms)

    if result.hand_landmarks:
        lm_list = result.hand_landmarks[0]

        # 2. Normalisation (identique a l'entrainement)
        vecteur = normaliser(lm_list).reshape(1, -1)

        # 3. Inference CNN
        probs     = model.predict(vecteur, verbose=0)[0]
        idx       = np.argmax(probs)
        confiance = probs[idx]
        geste     = CLASSES[idx] if confiance >= SEUIL_CONFIANCE
            else None

        # 4. Envoi si confiance suffisante
        if geste and geste != 'NEUTRAL':
```

```
            commande_queue.put_nowait({'type': geste})
```

6.2 Cooldowns anti-spam

Pour éviter l'envoi répété de commandes et les faux positifs de transition :

Commande	Cooldown	Rôle
LEFT, RIGHT	80 ms	Mouvement fluide continu
FIRE	80 ms	Tir fréquent (cooldown jeu : 500 ms)
ENTER	800 ms	Démarrage, une seule fois
NEUTRAL	—	Jamais envoyé

6.3 Correspondance des touches

Le serveur Node.js mappe les commandes texte en codes de touche standard :

Commande	Keycode	Touche
LEFT	37	Flèche gauche
RIGHT	39	Flèche droite
FIRE	32	Espace
ENTER	13	Entrée

7 Résultats, Limites et Analyse Critique

7.1 Performance en conditions réelles

Le réseau dense sur landmarks est **robuste en conditions réelles** : fond de pièce, éclairage variable, main de taille différente. Les gestes LEFT, RIGHT, ENTER et NEUTRAL sont reconnus sans erreur notable. La latence totale mesurée (capture → jeu) est de 40 à 80 ms, imperceptible.

7.2 Limites identifiées

1. FIRE partiellement entraîné. Les données de collecte pour le geste FIRE ont été capturées principalement avec la main gauche. En landmarks normalisés, la main droite effectuant le même geste produit un vecteur approximativement symétrique ($x_i \to -x_i$), qui constitue une forme différente pour le réseau. La classification FIRE est donc moins fiable avec la main droite.

2. Actions simultanées impossibles. La classification étant unimodale (une classe par frame), le système ne peut envoyer qu'une commande à la fois. Tirer (`FIRE`) pen-

dant un déplacement (`LEFT` ou `RIGHT`) est impossible avec l'architecture actuelle.

3. NEUTRAL difficile à distinguer. La classe NEUTRAL (main au repos) partage des configurations de landmarks proches avec FIRE (doigts semi-fléchis $\approx$ poing) et LEFT (position intermédiaire). Cela génère 2 des 2 erreurs de validation et quelques faux positifs en jeu réel.

8 Perspectives d'Amélioration

8.1 Court terme — Classes composites pour tirer en se déplaçant

La limitation jouabilité la plus impactante est l'impossibilité de tirer en se déplaçant. La solution la plus simple est d'ajouter 2 nouvelles classes d'entraînement :

Classe	Geste	Commandes envoyées
`LEFT_FIRE`	Poing incliné à gauche	LEFT + FIRE simultanés
`RIGHT_FIRE`	Poing incliné à droite	RIGHT + FIRE simultanés

La transition `LEFT` $\rightarrow$ `LEFT_FIRE` consiste uniquement à fermer le poing sans déplacer le bras — geste naturel, peu fatigant, et géométriquement très différent de tous les autres en landmarks.

8.2 Court terme — Robustesse FIRE deux mains

Collecter le geste FIRE avec les deux mains à parts égales lors de la prochaine session de collecte. Alternativement, générer les échantillons main droite par augmentation de données ($x_i \rightarrow -x_i$, symétrie axiale), sans recollecte.

8.3 Moyen terme — LSTM pour gestes dynamiques

Entraîner sur des séquences temporelles de landmarks (5 à 10 frames) plutôt que sur une frame isolée. Cela permettrait de reconnaître des *gestes dynamiques* (balayage de gauche à droite, fermeture rapide du poing) distincts des gestes statiques, réduisant les ambiguïtés géométriques.

Listing 14 – Architecture LSTM envisag ?e pour gestes dynamiques.

```
model = keras.Sequential([
    keras.Input(shape=(10, 63)),      # 10 frames x 63 coords
    keras.layers.LSTM(128, return_sequences=True),
    keras.layers.LSTM(64),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(n_classes, activation='softmax'),
])
```

8.4 Long terme — Deux mains simultanées

MediaPipe supporte jusqu'à 2 mains simultanément. En traitant les deux, on obtient naturellement 2 canaux de contrôle indépendants : par exemple, main gauche = direction continue, main droite = tir. Cela résout le problème de simultanéité de manière naturelle et intuitive.

9 Conclusion

Ce projet a permis de construire et déployer un pipeline complet de vision par ordinateur appliqué au jeu vidéo. Les conclusions techniques principales sont :

1. **La représentation prime sur la complexité du modèle.** Passer de 12 288 pixels à 63 landmarks normalisés a fait passer la précision de 34.3% à 99.6% avec un modèle 66 fois plus petit.
2. **La généralisation dépend des données, pas du dataset.** Un modèle entraîné sur 8 000 images à fond blanc (Kaggle) ne fonctionne pas en conditions réelles. Nos 2 100 landmarks, collectés dans notre propre environnement, donnent un modèle robuste et déployable.
3. **L'ingénierie système est aussi importante que le modèle.** Le threading Python, le hold timer Node.js et les cooldowns sont des décisions d'ingénierie — sans elles, un bon modèle donnerait une mauvaise expérience utilisateur.
4. **Les limites sont comprises et documentées.** FIRE moins robuste avec la main droite, impossibilité de tirer en se déplaçant — ces limitations sont connues, analysées et des solutions concrètes sont proposées.

A Structure du Dépôt

```
projet_computer_vision/
|-- index.html                  # Jeu Space Invaders
|-- comment_jouer.html          # Guide interactif des gestes
|-- server.js                   # Serveur WebSocket Node.js
|-- lancer_jeu.ps1              # Script de demarrage automatique
|-- hand_landmarker.task        # Modele MediaPipe pre-entraine
|-- game.bundle.js / game.js    # Jeu JavaScript
|-- core/ entities/ utils/      # Code source du jeu
+-- cnn_training/
    |-- 1_collecter_images.py       # Collecte images brutes
    |-- 2_entrainer_cnn.py          # CNN sur images (approche 1)
    |-- 3_collecter_landmarks.py    # Collecte landmarks MediaPipe
    |-- 4_entrainer_landmarks.py    # Reseau dense sur landmarks
    |-- 5_jouer_avec_cnn.py         # Script de jeu (solution finale)
    |-- test_cnn_local.py           # Test temps reel sans jeu
    +-- modeles/
        |-- gestes_landmarks.keras  # Modele final (99.6%)
        |-- classes_landmarks.txt   # Liste des 5 classes
        |-- courbes_landmarks.png   # Courbes d'entrainement
        +-- images_rapport/         # Photos des gestes
```

B Guide de Lancement

B.1 Prérequis

```
pip install tensorflow mediapipe opencv-python websocket-client
npm install   # dans le dossier du projet
```

B.2 Lancement automatique (recommande)

```
cd C:\Users\...\projet_computer_vision
.\lancer_jeu.ps1
```

Ouvre automatiquement les 3 processus et lance la webcam.

B.3 Lancement manuel

```
# Terminal 1
node server.js
```

```
# Terminal 2
python -m http.server 8000

# Terminal 3
cd cnn_training && python 5_jouer_avec_cnn.py

# Navigateur : http://localhost:8000
```

B.4 Test du modèle seul

```
cd cnn_training
python test_cnn_local.py
```